

Chapter 4

Classification (Part 2)

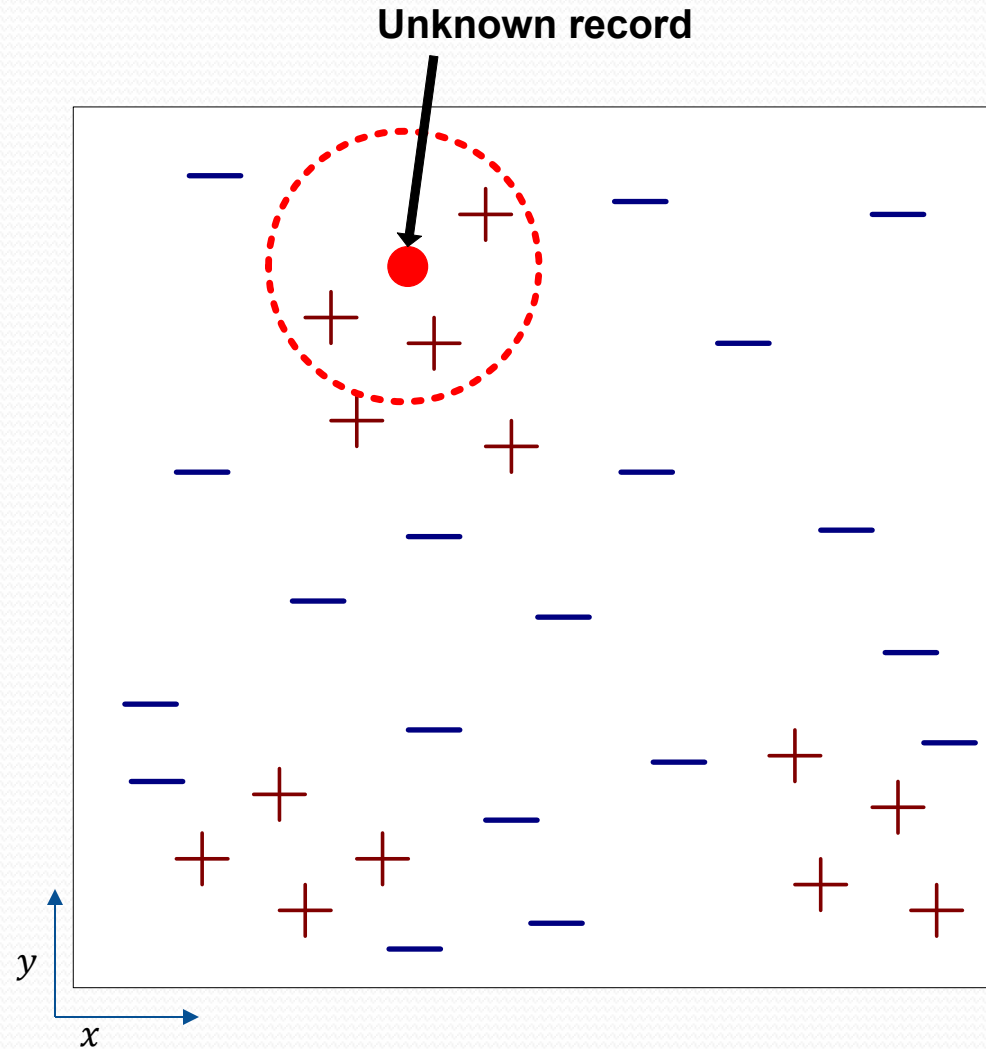
Overview

- Nearest-neighbor classifiers
- Bayesian classifiers
- Support vector machine (SVM)
- Logistic regression
- Ensemble methods
 - Bagging, Boosting, Random Forests
- Reading assignment:
 - Sections 4.3, 4.4 (208-224); 4.6 (243-245); 4.9 (276-286); 4.10 (296-306; 310-313).

Nearest Neighbor Classifiers

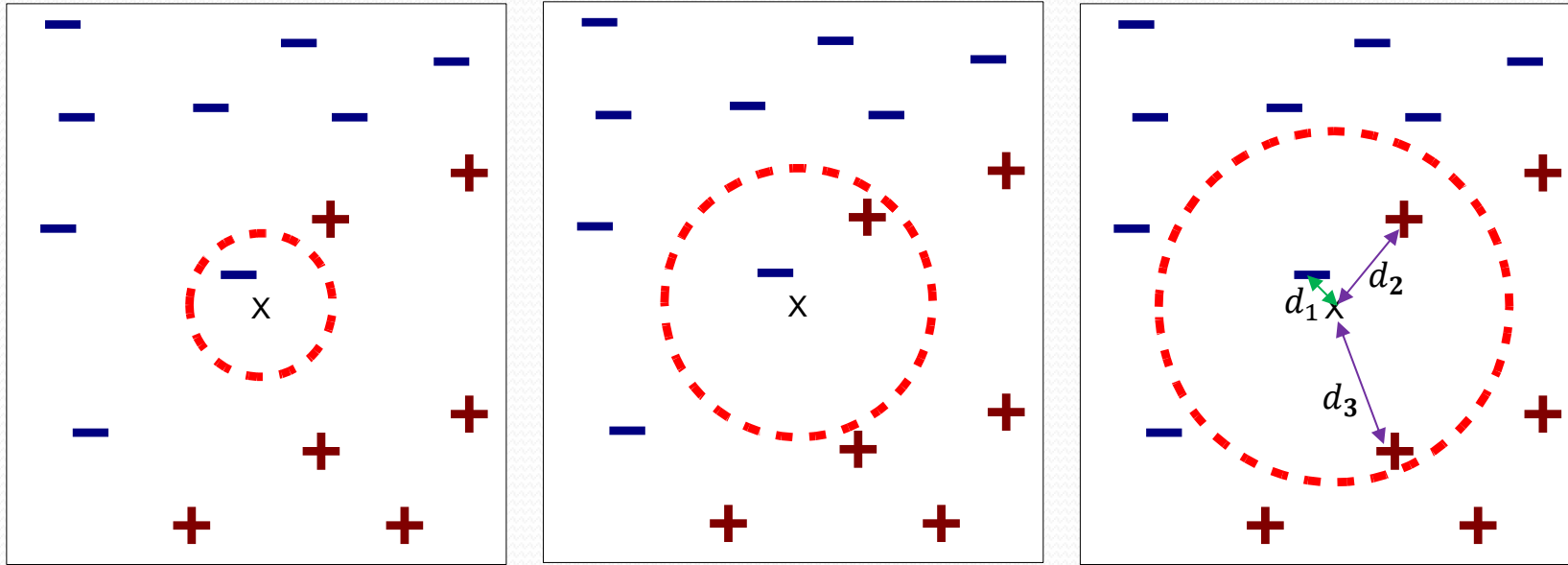
- Basic idea:
 - Given an unlabeled record Y , find the records in the training set that are most similar to Y (the *nearest neighbors*) to infer the label of Y .

Nearest-Neighbor Classifiers



- Requires three things
 - A set of stored *labeled records*
 - A *distance measure* to compute distance between records
 - The value of *k*, the number of nearest neighbors to retrieve
- To classify an unknown record:
 - Compute distances to other records with known labels
 - Identify *k* nearest neighbors
 - Use class labels of nearest neighbors to determine the class label of unknown record (e.g., by taking majority vote)

Definition of Nearest Neighbor



(a) 1-nearest neighbor

(b) 2-nearest neighbor

(c) 3-nearest neighbor

k -nearest neighbors of a record x are data points that have the k smallest distances to x

Nearest Neighbor Classification

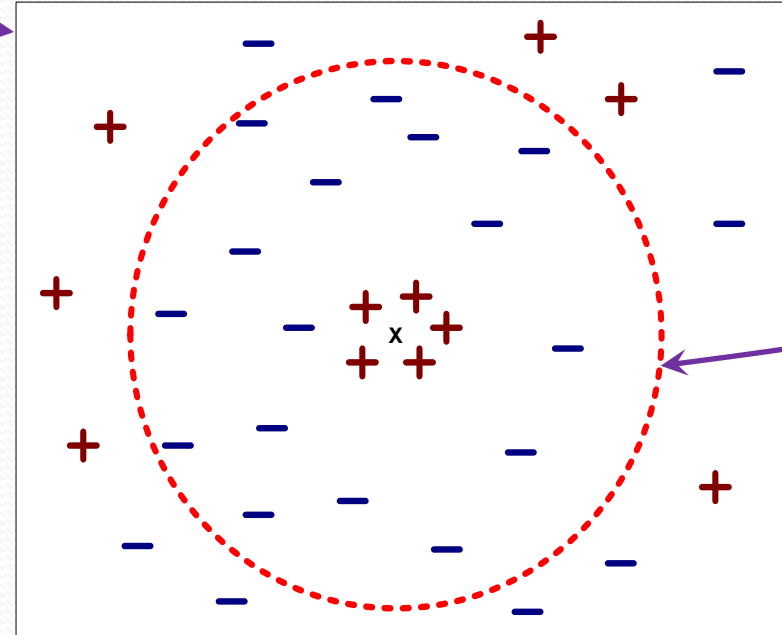
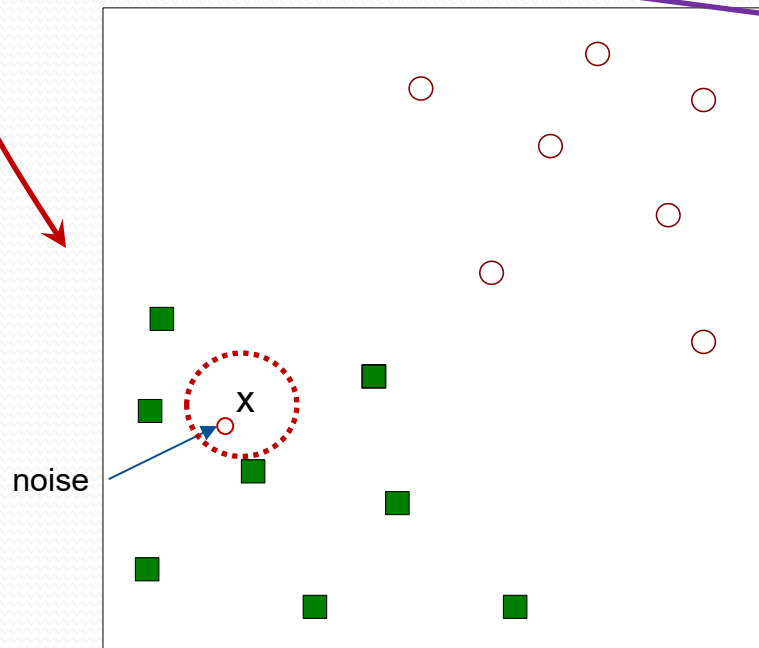
- Compute distance between two points:
 - E.g., Euclidean distance

$$\text{Objects } p, q \quad \underbrace{d(p, q)}_{\text{Euclidean distance}} = \sqrt{\sum_i \underbrace{(p_i - q_i)^2}_{i\text{-th attribute of Objects } p, q}}$$

- Determine the class from nearest neighbor list
 - take the majority vote of class labels among the k-nearest neighbors
 - We can also weigh the votes according to neighbors' distances
 - example weight factor, $w = 1/d^2$ (A closer neighbor contributes more votes.)
- Attributes must be normalized.

Nearest Neighbor Classification

- Choosing the value of k :
 - If k is too small: (a) sensitive to noise points; (b) overfitting.
 - If k is too large, neighborhood may include points from other classes



Neighborhood boundary crosses class boundary

Nearest Neighbor Classification

- k -NN classifiers are *lazy learners*
 - They do not build model explicitly beforehand
 - Avoid expensive model-building
 - K -NN search could be expensive
 - K -NN search is typically assisted by indices (e.g., k-d tree)
- Distance-based so it performs poorly in high-dimensional spaces.
- Feature selection is important.
 - Highly-correlated features shouldn't be all included in the distance function.
 - Irrelevant features should be removed.

Bayesian Classifier

- Based on Bayes' Theorem
- Given a hypothesis H and an observation X , denote $P(H|X)$ as the probability that the hypothesis H is true given X happens.
- Example:
 - H = an object O is an *apple*
 - X = an object O is *red* and *round*
 - $P(H|X)$ = prob. that an object O is an *apple* given that O is *red* and *round*

Bayes' Theorem

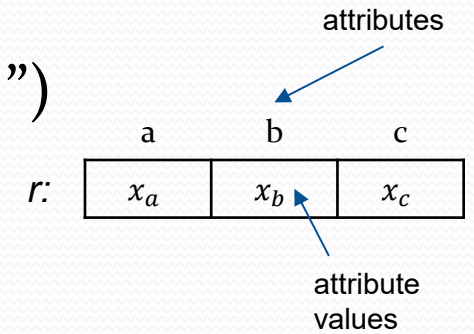
- Note that we can consider
 - $P(H)$ = probability that an arbitrary object is an apple
 - $P(X)$ = probability that an arbitrary object is red and round
 - $P(X|H)$ = probability that an object O is red and round given that O is an apple
 - $P(H|X)$ = probability that an object O is an apple given that O is red and round

Bayes' Theorem

- $P(H|X) = P(H, X) / P(X)$
- $P(X|H) = P(H, X) / P(H) \Rightarrow (P(H, X) = P(X|H) \cdot P(H))$
- $P(H|X) = P(X|H) * P(H) / P(X)$

Applying Bayes theorem to classification

- Given an unlabeled record r (to be classified), we consider
 - H : r is of a certain class, say, C_1 (denote that hypothesis H by “ C_1 ”)
 - X : the observation that a record shares r 's attribute values
- $P(C_1)$ = probability that a record is of class C_1
- $P(X)$ = probability that a record shares r 's attribute values
- $P(X|C_1)$ = probability that a record shares r 's attribute values given that the record is labeled C_1
- $P(C_1|X)$ = probability that a record is labeled C_1 given that it shares r 's attribute values



Applying Bayes' theorem to classification

- Suppose there are m class labels: $C_1, C_2, \dots, C_m$
- We want to determine which class record r should belong
- Method: compare $P(C_1|X), P(C_2|X), \dots, P(C_m|X)$ and pick the C_i with the largest probability

Applying Bayes' theorem to classification

- Note that:

- $P(C_1|X) = P(X|C_1) * P(C_1) / P(X); P(C_2|X) = P(X|C_2) * P(C_2) / P(X);$
- $P(C_1|X) > P(C_2|X) \Leftrightarrow P(X|C_1)P(C_1) > P(X|C_2)P(C_2)$
- Hence, we pick the class C_i with the largest value of
 - $P(X|C_i)P(C_i)$
- Given a training set D , we can estimate $P(C_i)$ by n_i/N , where
 - n_i = number of records in D that are of class C_i , and
 - N = total number of records in D

of records in DB that are of class C_i and match r 's values
 n_i

Naïve Bayesian Classification

- $P(X|C_i)$, however, is difficult to estimate
- Naïve Bayesian Classification assumes that the values of the attributes are **conditionally independent** of one another.
- That is,

$$P(X|C_i) = \prod_{k=1}^m P(x_k|C_i)$$

x_k = the k -th attribute's value of a record r

	height	weight	age	gender	income
Attribute:	1	2	3	...	m
X :	x_1	x_2	x_3		x_m

Record r

Naïve Bayesian Classification

- If Attribute k is categorical (e.g., nominal, ordinal), then $P(x_k|C_i)$ can be estimated by counting

$$n_{ik} / n_i$$

where n_{ik} = number of records in the dataset that are of class C_i and whose values for attribute k is x_k

Among all the records that are of class C_i ,
how many of them are the same as r w.r.t. the k -th attribute (i.e., have attribute value x_k)

Record id	Age	Income	Student	Credit-rating	Own-computer
1	< 30	High	No	Bad	No
2	< 30	High	No	Good	No
3	30 .. 40	High	No	Bad	Yes
4	> 40	Medium	No	Bad	Yes
5	>40	Low	Yes	Bad	Yes
6	> 40	Low	Yes	Good	No
7	30 .. 40	Low	Yes	Good	Yes
8	< 30	Medium	No	Bad	No
9	< 30	Low	Yes	Bad	Yes
10	> 40	Medium	Yes	Bad	Yes
11	< 30	Medium	Yes	Good	Yes
12	30 .. 40	Medium	No	Good	Yes
13	30 .. 40	High	Yes	Bad	Yes
14	> 40	Medium	No	Good	No

Example

- Given a record X :

<i>Age</i>	<i>Income</i>	<i>Student</i>	<i>Credit-rating</i>
< 30	medium	yes	bad

is X a computer-owner or not?

Note: this record is not found in the dataset.
Hence, $P(X|C_i)$ cannot be directly computed.

Example

- 2 classes:
 - $C_1 = \text{"O.C. = yes"}; P(C_1) = 9/14$
 - $C_2 = \text{"O.C. = no"}; P(C_2) = 5/14$
- 4 attributes:
 - $P(\text{Age} < 30 \mid C_1) = 2/9$
 - $P(\text{Income} = \text{medium} \mid C_1) = 4/9$
 - $P(\text{Student} = \text{yes} \mid C_1) = 6/9$
 - $P(\text{C.R} = \text{bad} \mid C_1) = 6/9$
- Hence,
 - $P(X|C_1) = (2/9)(4/9)(6/9)(6/9) = 0.044$

Example

- Similarly, we have:
 - $P(X|C_2) = 0.019$
- Therefore,
 - $P(X|C_1)P(C_1) = (9/14) * 0.044 = \mathbf{0.028}$
 - $P(X|C_2)P(C_2) = (5/14) * 0.019 = 0.007$
- X is classified as C_1 , or X is a computer-owner

What about numerical attributes?

- For numerical attributes: How to estimate $P(x_k|C_i)$?
 - Discretize the range into bins
 - thus converting the attribute into nominal
 - results are sensitive to discretization
 - Probability density estimation:
 - Assume the numerical attribute values follow a normal distribution
 - Use data to estimate parameters of distribution (e.g., mean and standard deviation)
 - With the probability distribution, estimate the conditional *probability density* $P'(x_k|C_i)$
 - Compare classes based on their probability densities.

Numerical attributes

Numerical attribute

Class attribute

Tid	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

mean of attribute i among objects of class C_j

- Normal distribution:

$$P(x_i|C_j) = \frac{1}{\sqrt{2\pi\sigma_{ij}^2}} e^{-\frac{(x_i - \mu_{ij})^2}{2\sigma_{ij}^2}}$$

standard deviation

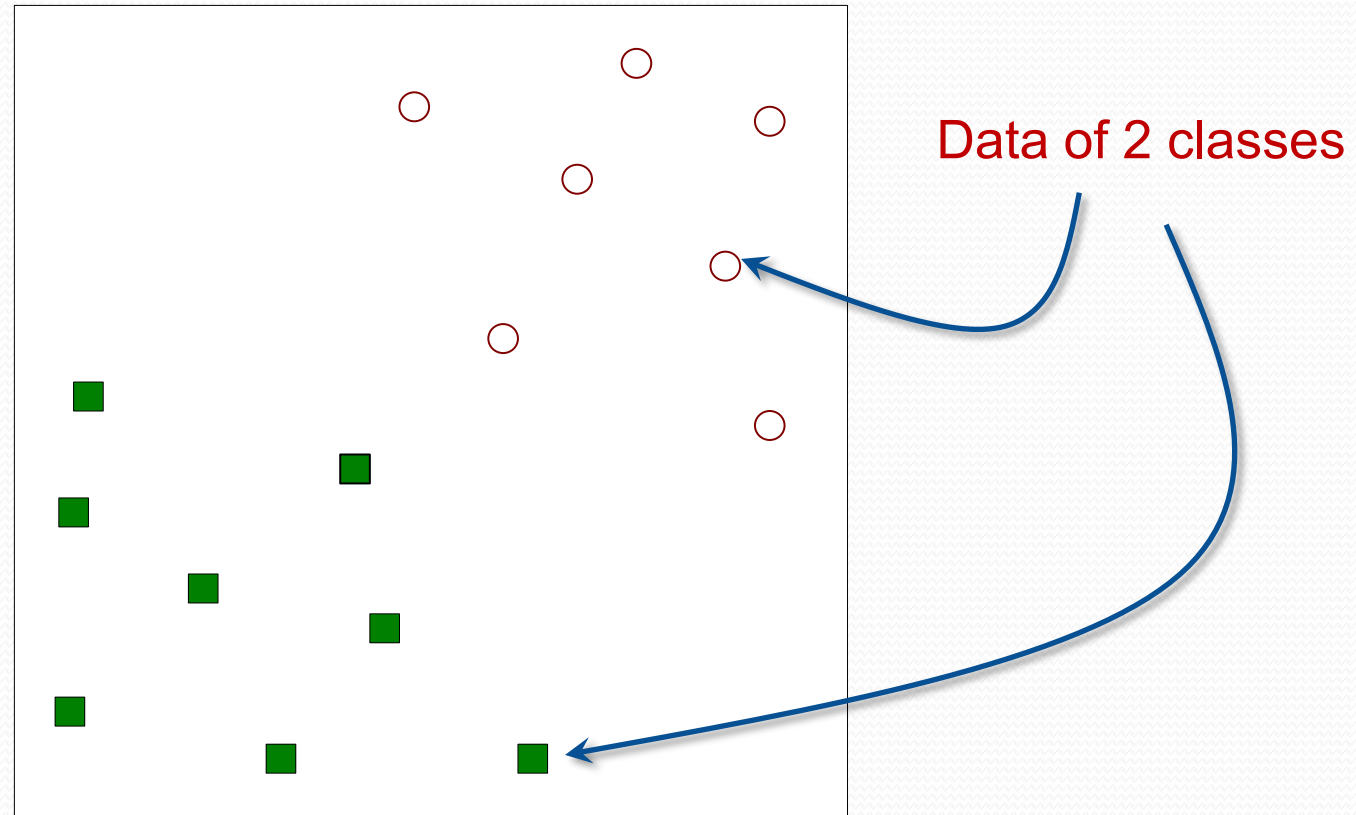
- For each pair of numerical attribute i and class C_j
- For (Income, Class=No):
 - If Class=No
 - sample mean (μ) = 110
 - sample variance (σ^2) = 2975

$$P'(\text{Income} = 120|\text{No}) = \frac{1}{\sqrt{2\pi(2975)}} e^{-\frac{(120-110)^2}{2(2975)}} = 0.0072$$

Naïve Bayesian Classification (Summary)

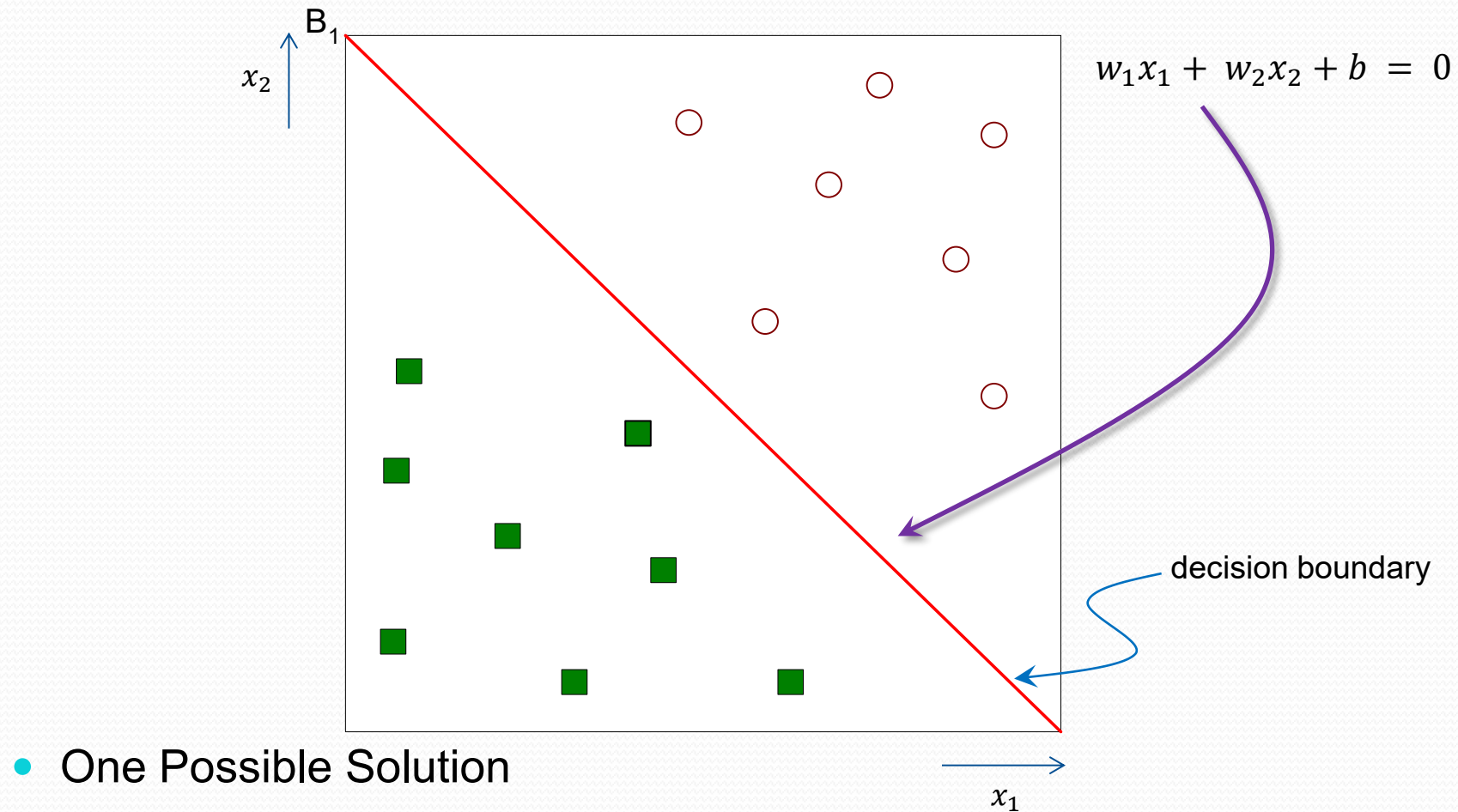
- Based on statistics of the whole dataset $\Rightarrow$ robust to isolated noise points.
- In contrast, kNN classifier depends only on a few data objects (the “*neighbors*”) to make prediction $\Rightarrow$ susceptible to noise and overfitting.
- Independence assumption may not hold
 - Use other techniques such as Bayesian Belief Networks (BBN) to capture attributes correlation

Support Vector Machines (SVM)

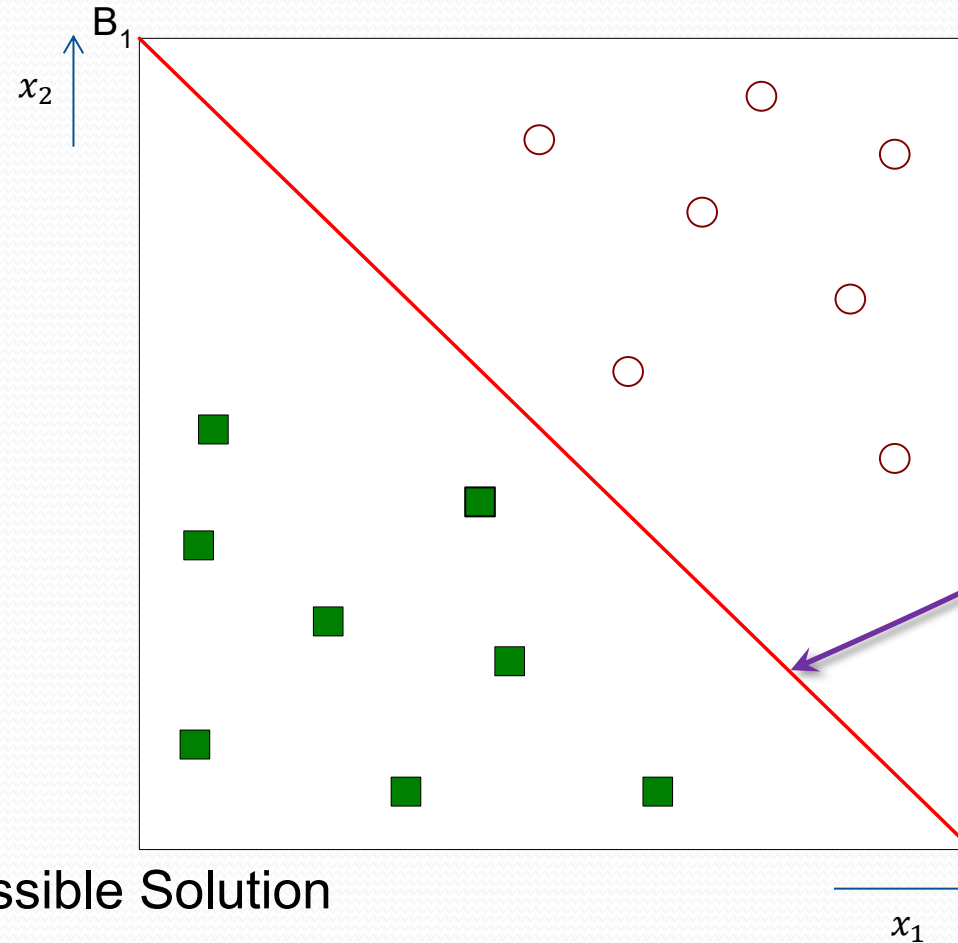


- Find a linear hyperplane (decision boundary) that separates the data into their respective classes

Support Vector Machines



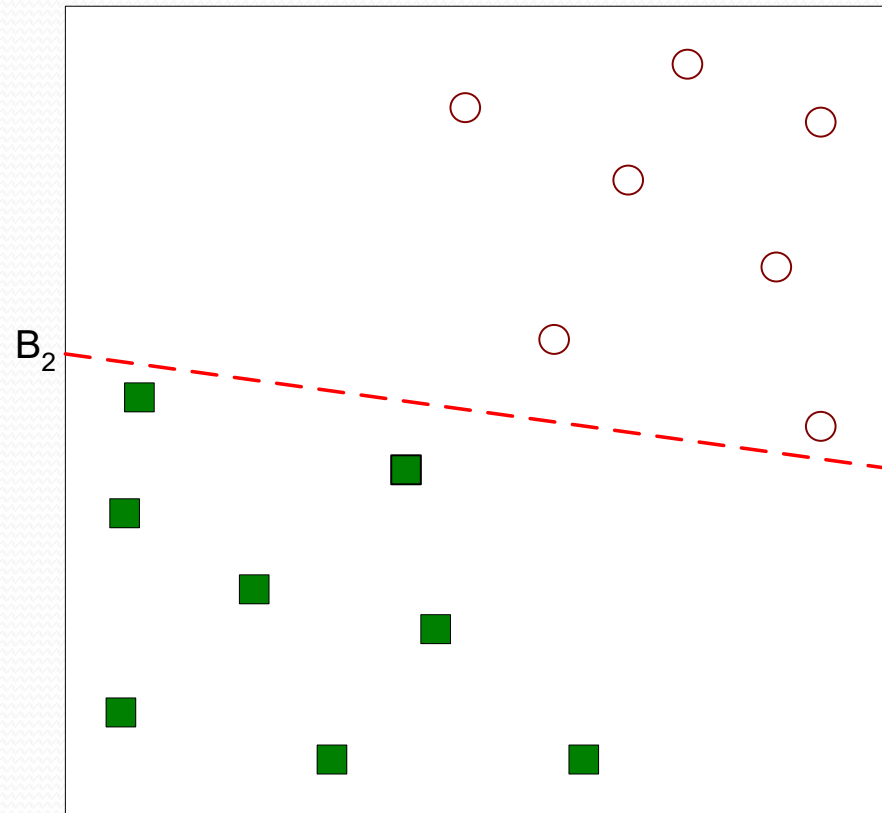
Support Vector Machines



$$\begin{aligned}\vec{w} \cdot \vec{x} + b &= 0 \\ \vec{w} &= (w_1, w_2); \\ \vec{x} &= (x_1, x_2)^T\end{aligned}$$

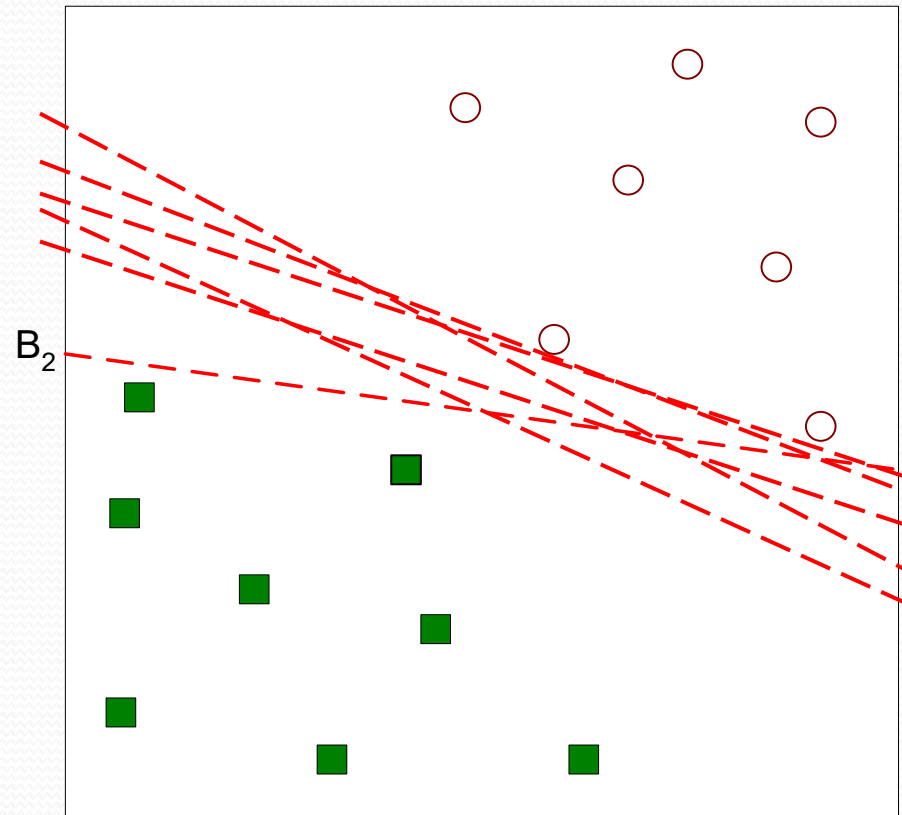
- One Possible Solution

Support Vector Machines



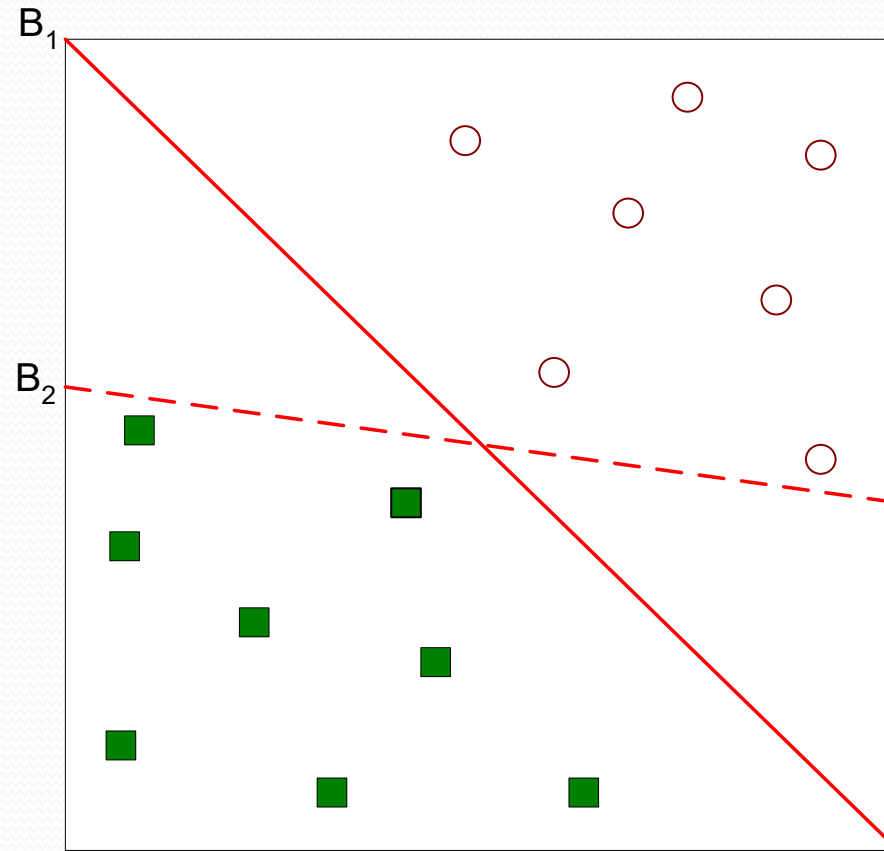
- Another possible solution

Support Vector Machines



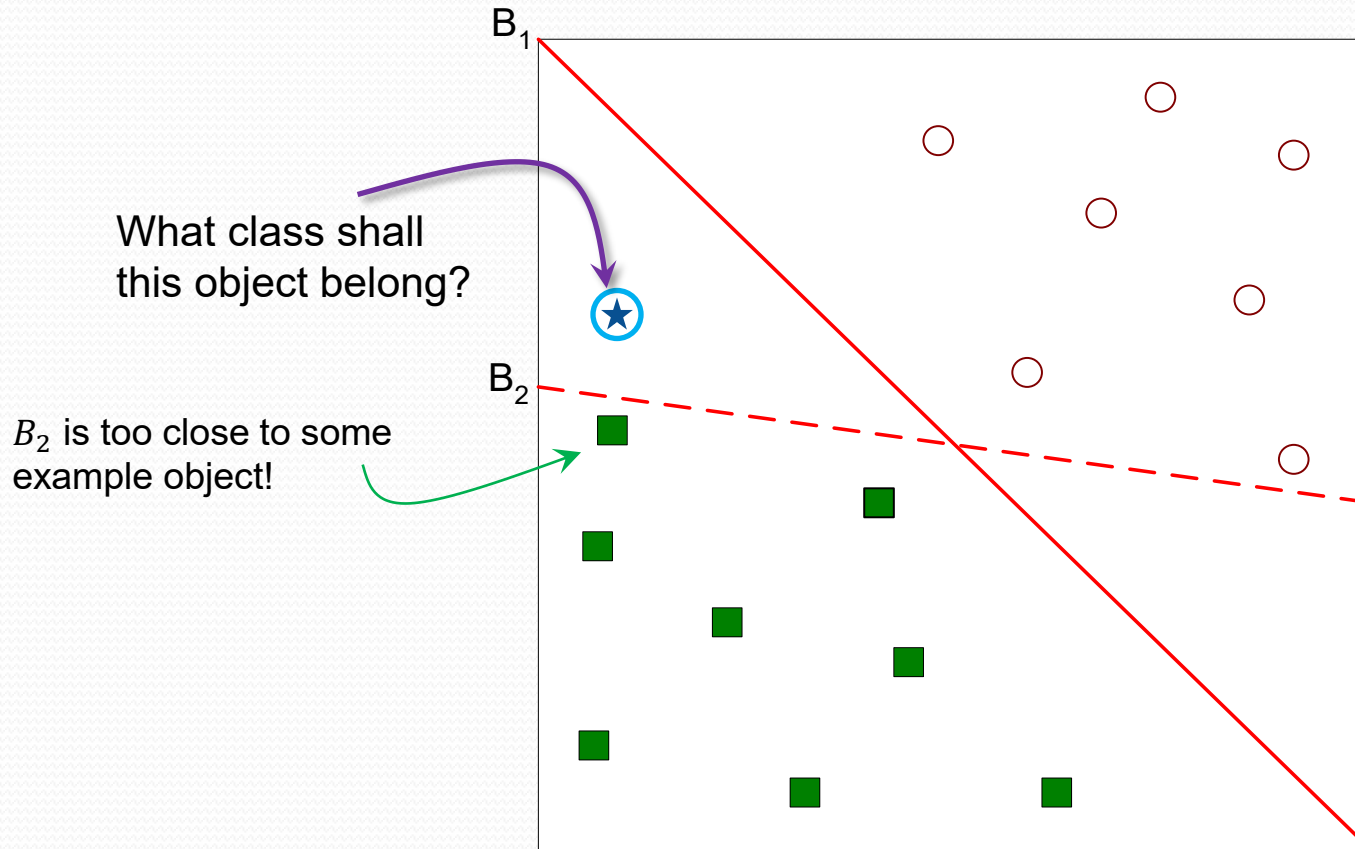
- Other possible solutions

Support Vector Machines

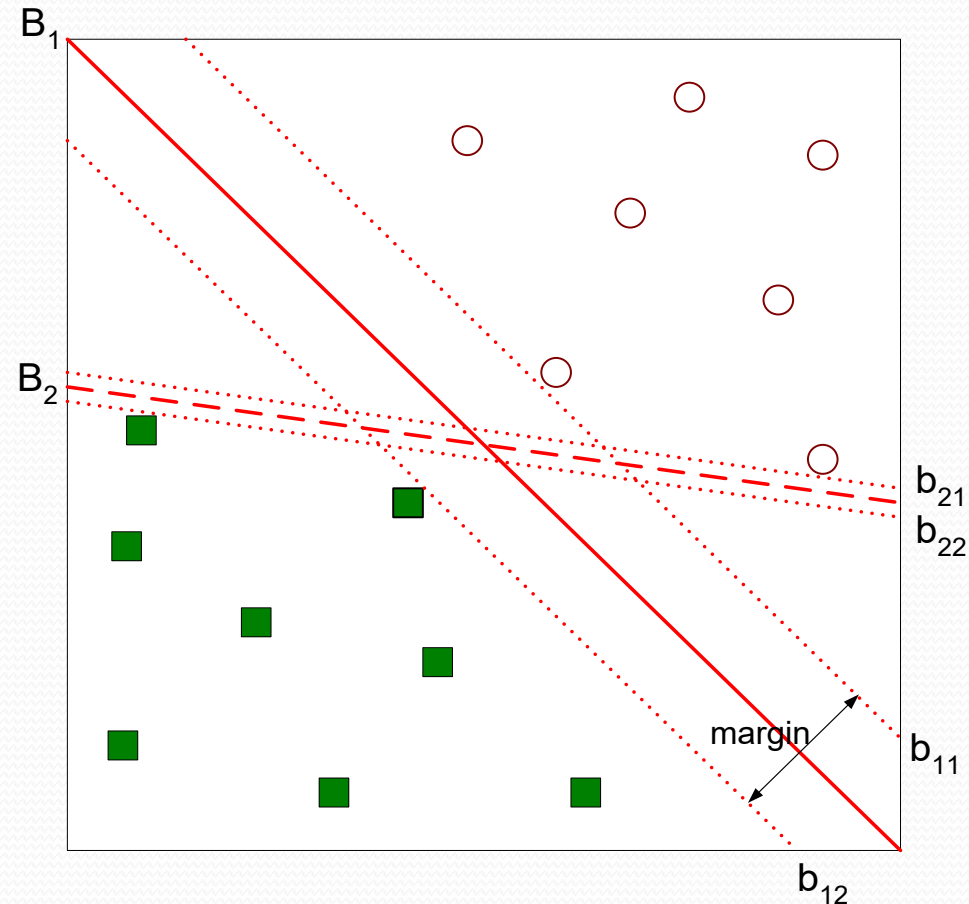


- Which one is better? B_1 or B_2 ?
- How shall we define quality?

Support Vector Machines

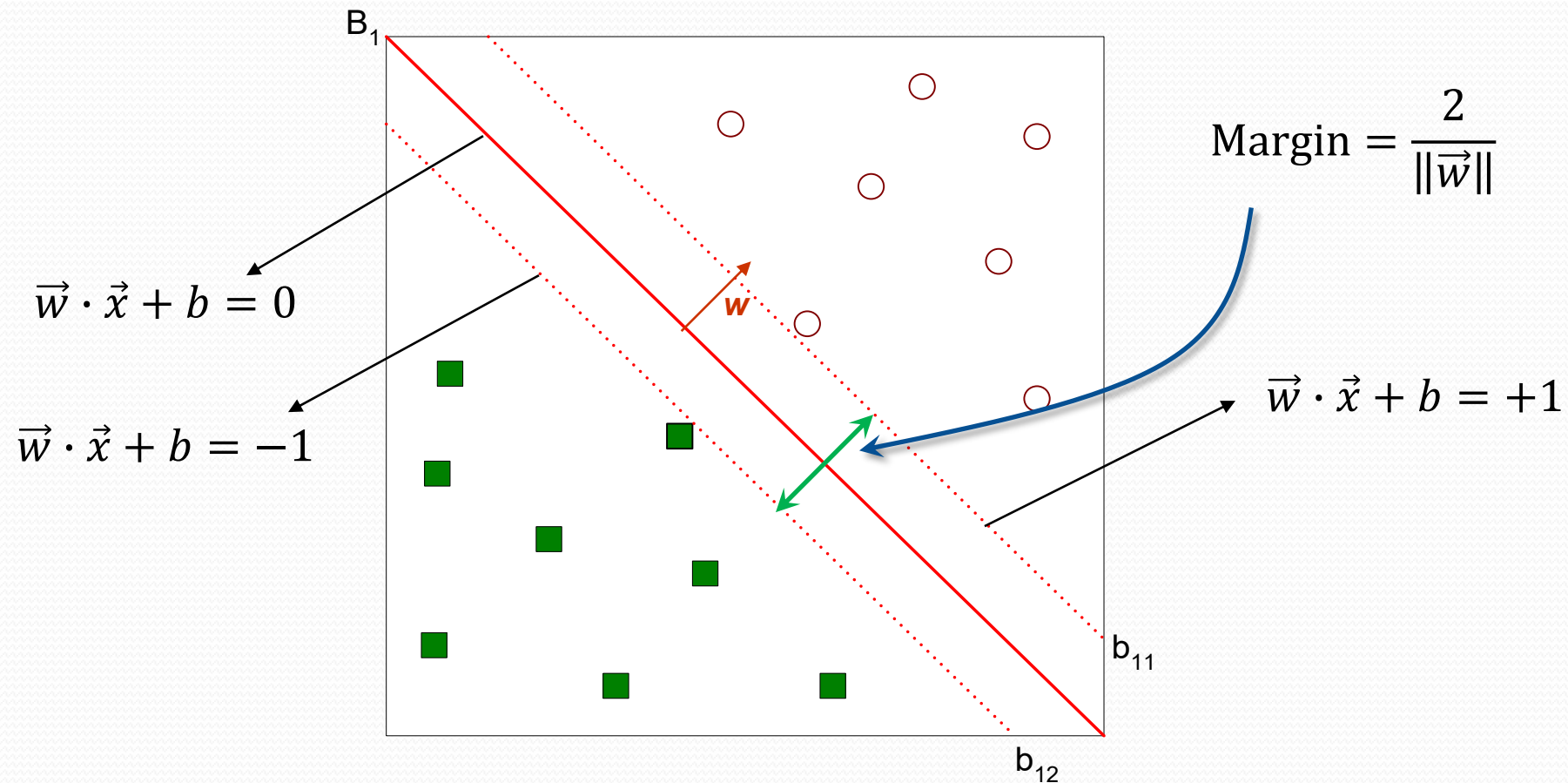


Support Vector Machines



- Find hyperplane that *maximizes* the margin $\Rightarrow B_1$ is better than B_2

Support Vector Machines

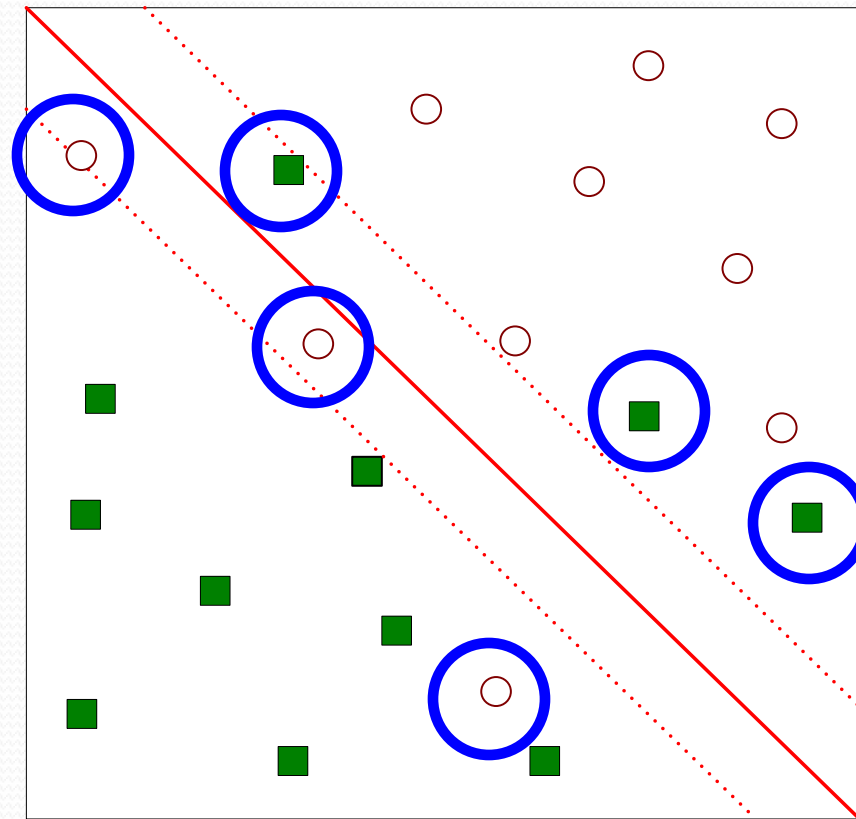


Support Vector Machines

- We want to maximize: $\text{Margin} = \frac{2}{\|\vec{w}\|}$
- Which is equivalent to minimizing: $L(w) = \frac{1}{2} \|\vec{w}\|^2$
 *loss function*
- Subject to the following constraints:
 - For all $\vec{x}$ in training set,
$$\text{if } \text{label}(\vec{x}) = \begin{cases} 1 & \text{then } \vec{w} \cdot \vec{x} + b \geq 1 \\ -1 & \text{then } \vec{w} \cdot \vec{x} + b \leq -1 \end{cases}$$

- This is a constrained optimization problem
 - Numerical approaches to solve it (quadratic programming)

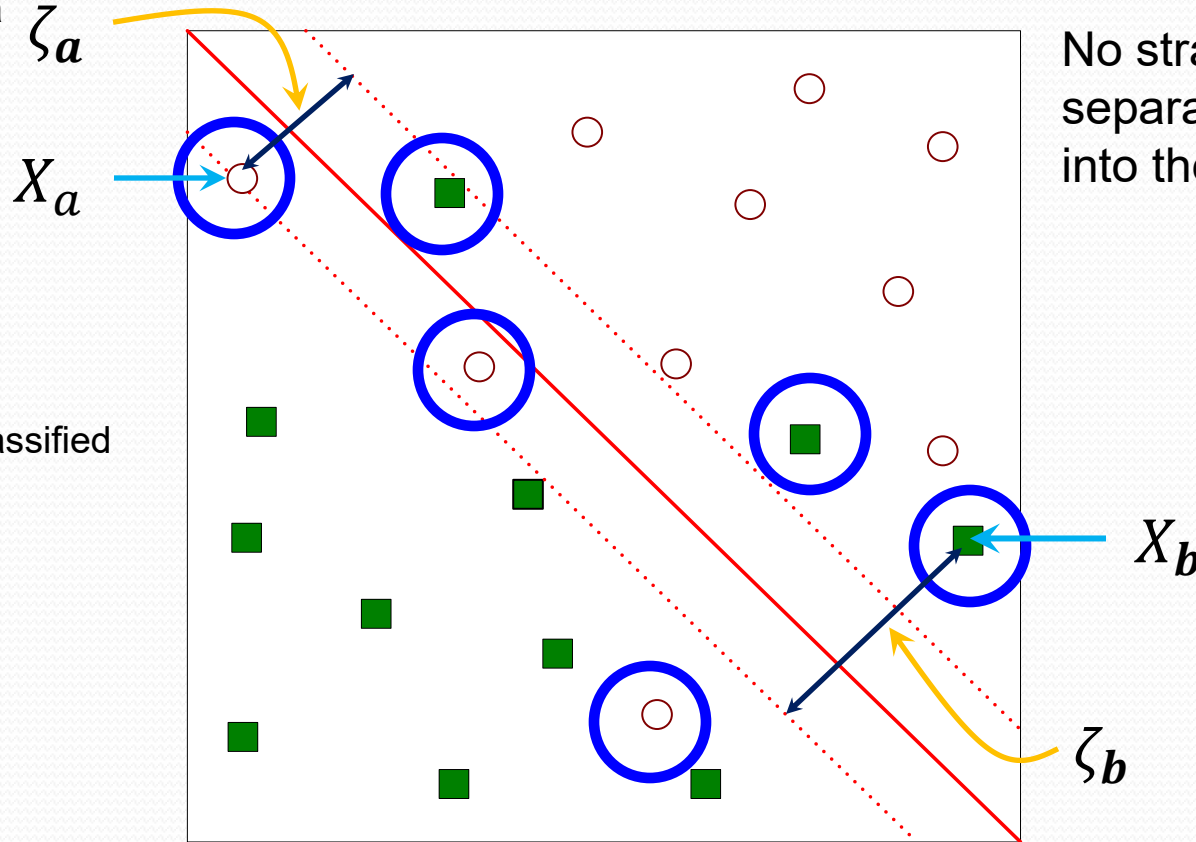
What if the problem is not linearly separable?



No straight line can separate the examples into their classes

What if the problem is not linearly separable?

Each point is given a
slack variable



No straight line can
separate the examples
into their classes

Idea:
Allow some objects to be misclassified
but penalize the model by
the total amount of slack.

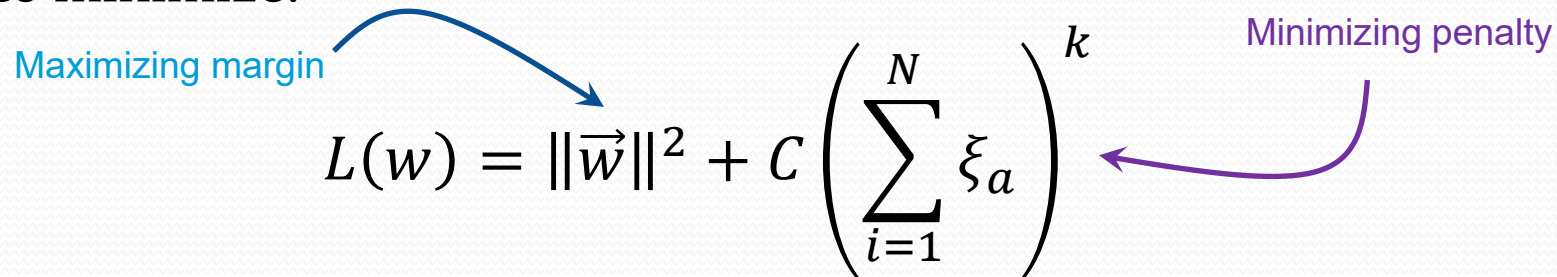
Support Vector Machines

- What if the problem is not linearly separable?
 - Introduce slack variables. Each labeled record $\vec{x}_a$ is given a slack variable ξ_a
 - Need to minimize:

Maximizing margin

$$L(w) = \|\vec{w}\|^2 + C \left(\sum_{i=1}^N \xi_a \right)^k$$

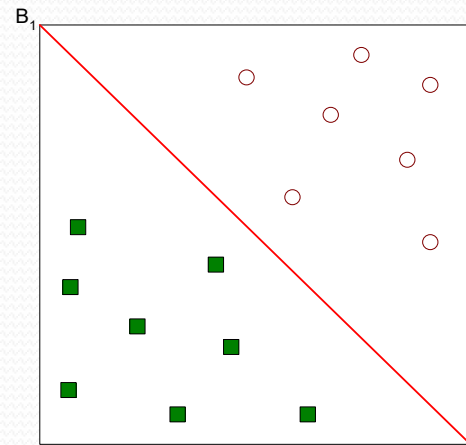
Minimizing penalty



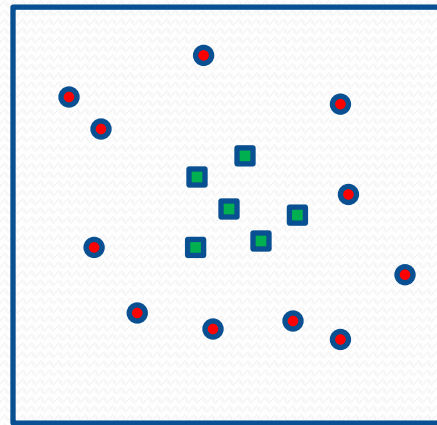
- Subject to:

$$\text{if } \text{label}(\vec{x}_a) = \begin{cases} 1 & \text{then } \vec{w} \cdot \vec{x}_a + b \geq 1 - \xi_a \\ -1 & \text{then } \vec{w} \cdot \vec{x}_a + b \leq -1 + \xi_a \end{cases}$$

The Kernel Trick



Linearly separable

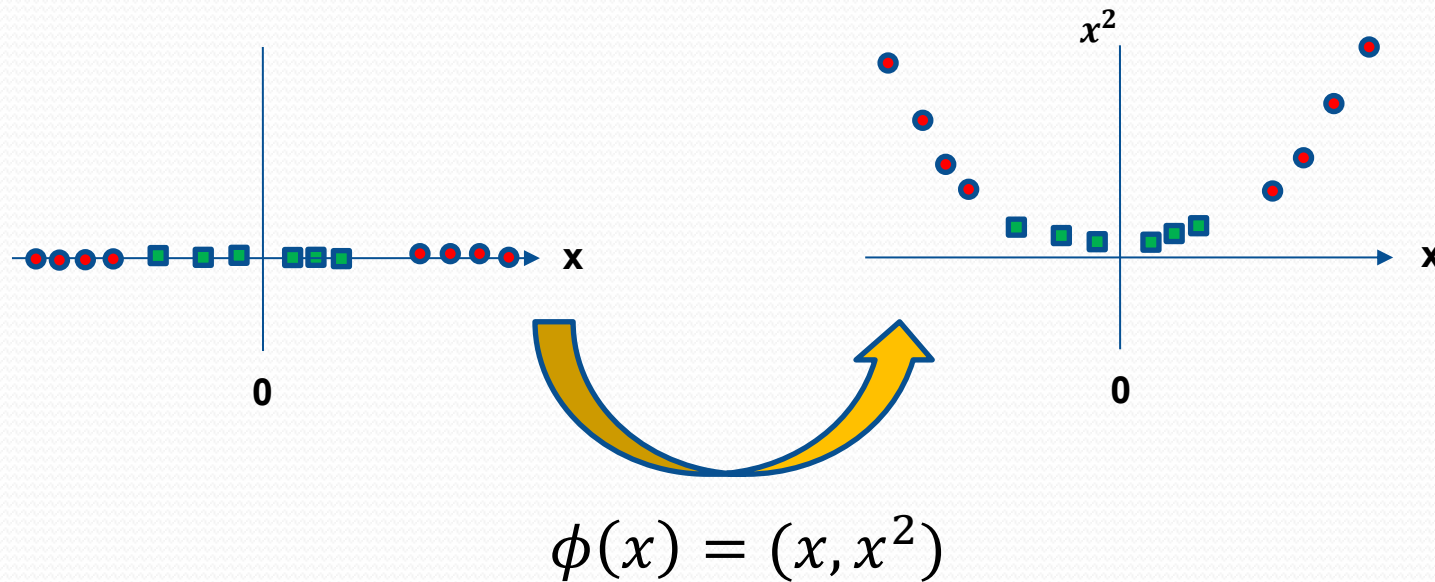


Not linearly separable



The Kernel Trick

- Transform the data from a low-dimensional space (e.g., x) to a higher-dimensional space ($\phi(x)$) so that the data is more likely to be linearly separable.



The Kernel Trick

- In SVM, computing the support vectors (or equivalently the hyperplane) requires computing many “dot products” (or similarity) of data points:
 $\vec{x} \cdot \vec{y} \quad (\vec{x}, \vec{y} \in \text{dataset}).$
- ϕ is called a *kernel* if $\phi(\vec{x}) \cdot \phi(\vec{y})$ can be computed by a function $K(\vec{x} \cdot \vec{y})$.
That is, the computation can be done in the original (low-d) space.

Similarity in original space

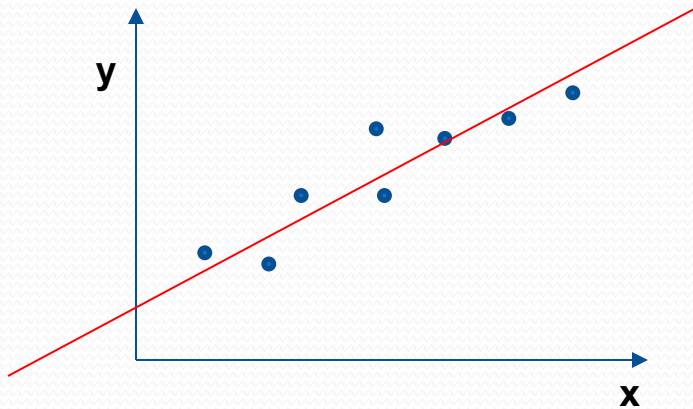
Similarity in higher space

The Kernel Trick

- Example: $\vec{x} = (x_1, x_2) \rightarrow \phi(\vec{x}) = (x_1^2, x_2^2, \sqrt{2} x_1 x_2)$
- $\phi(\vec{x}) \cdot \phi(\vec{y})$
 - $= (x_1^2, x_2^2, \sqrt{2} x_1 x_2) \cdot (y_1^2, y_2^2, \sqrt{2} y_1 y_2)$
 - $= x_1^2 y_1^2 + 2x_1 x_2 y_1 y_2 + x_2^2 y_2^2$
 - $= (x_1 y_1 + x_2 y_2)^2$
 - $= (\vec{x} \cdot \vec{y})^2$

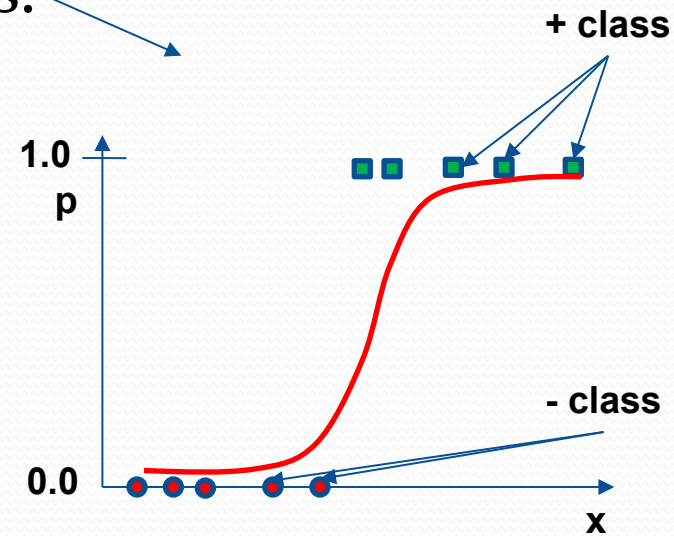
Logistic Regression (Idea)

- Use regression analysis to perform classification.
- Consider a 2-class problem: +/- classes.



Linear regression: predict a *numerical* dependent variable y from an independent variable x

Fit a *line* to the examples

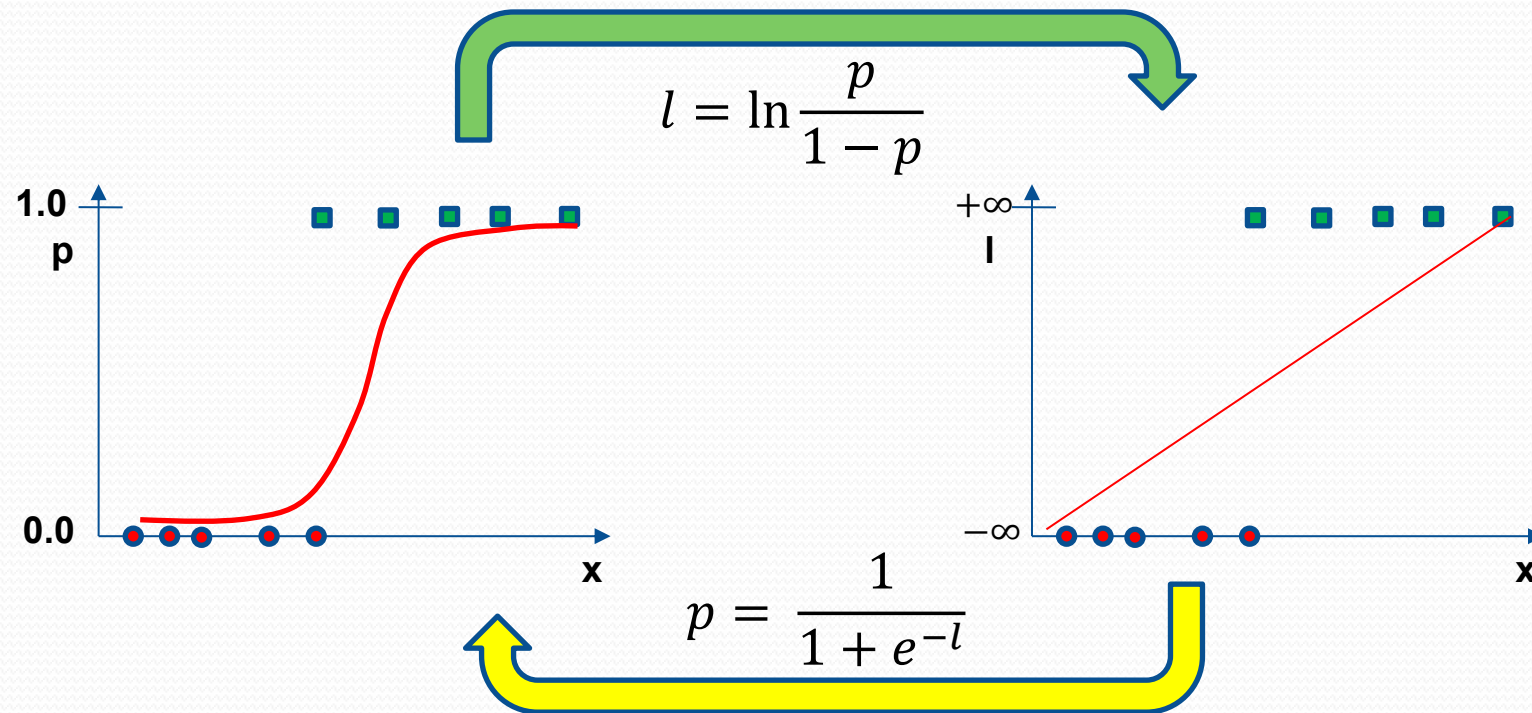


Logistic regression: predict the *probability of positive class* from an independent variable x

Fit a *logistic regression curve* to the examples

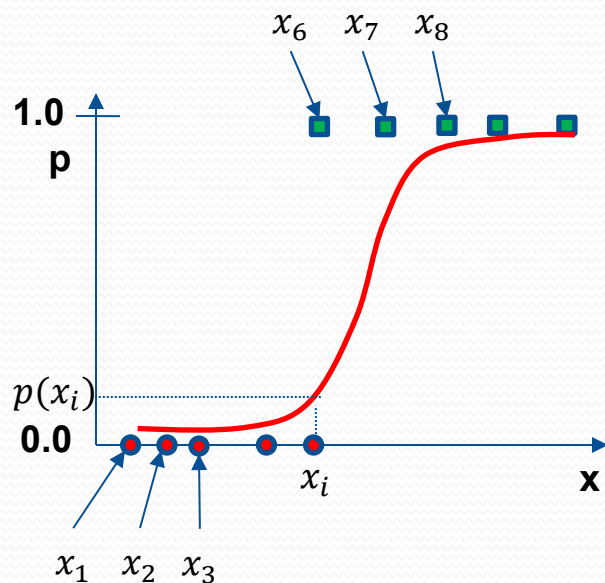
Logistic Regression

- Assume a linear relation between *log-odds* (l) and independent variables.



Logistic Regression

- Fit curve by maximizing likelihood

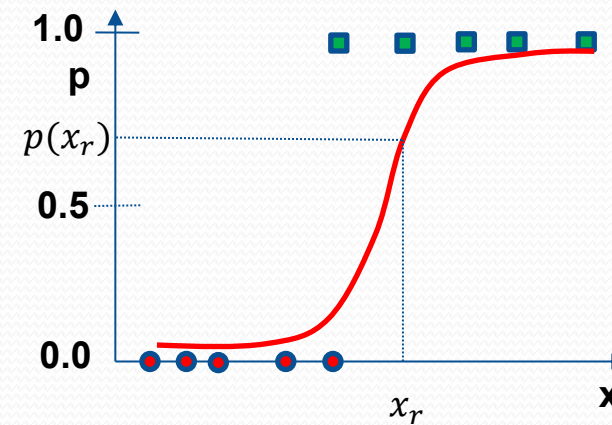


- “-” class: x_1, x_2, x_3, x_4, x_5 ●
- “+” class: $x_6, x_7, x_8, x_9, x_{10}$ ■
- $p(x_i)$ = prob that x_i is “+”
 $1 - p(x_i)$ = prob that x_i is “-”
- Likelihood of the model:

$$\underbrace{\prod_{i=1}^5 (1 - p(x_i))}_{\text{-ve class objects}} \underbrace{\prod_{i=6}^{10} p(x_i)}_{\text{+ve class objects}}$$

Logistic Regression

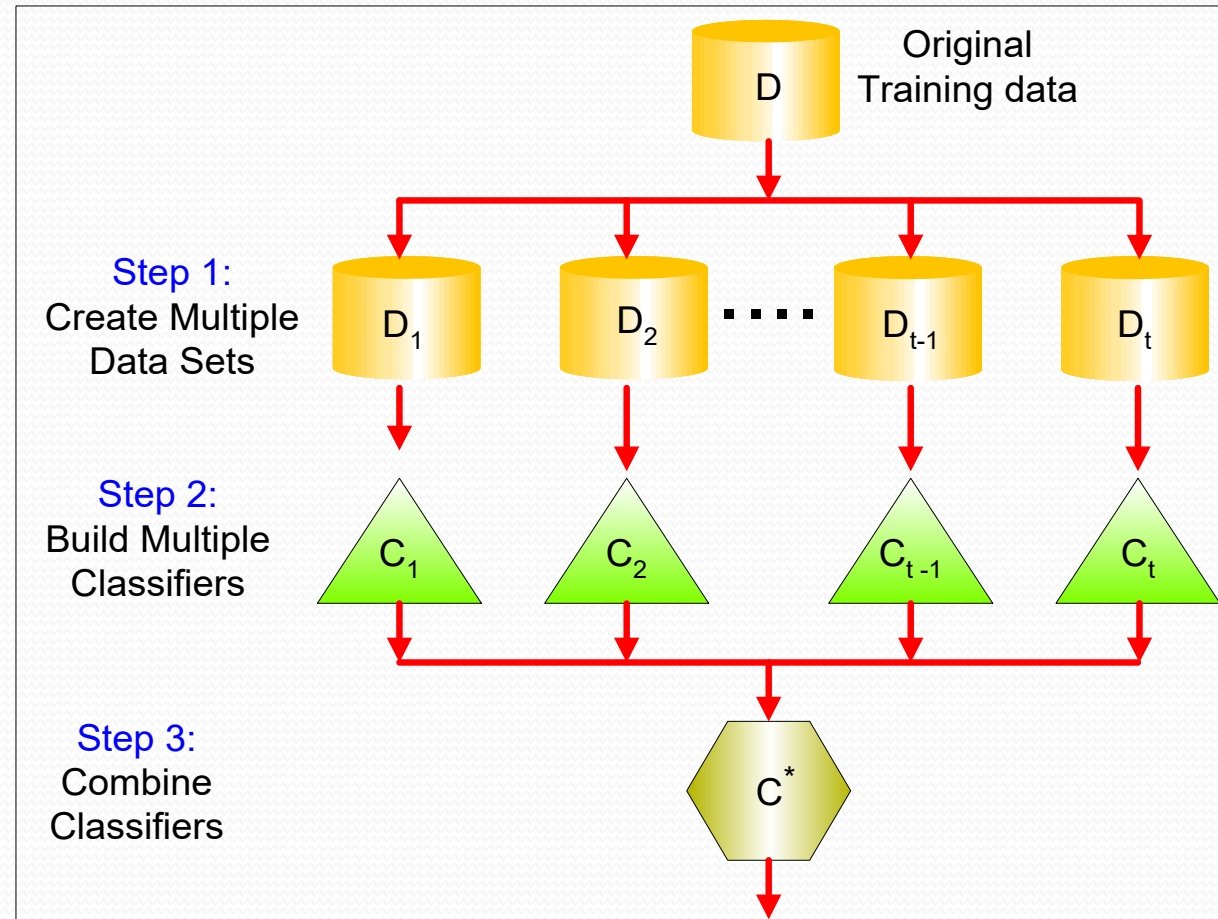
- Given a record r with unknown label. Let x of $r = x_r$. Classify r as “+” if $p(x_r) \geq t$, where t is a cut-off threshold, e.g., 0.5.



Ensemble Methods

- Construct a set of classifiers from the training data
- Predict class label of previously unseen records by aggregating predictions made by multiple classifiers
 - voting

General Idea



Examples of Ensemble Methods

- How to generate an ensemble of classifiers?
 - use different training sets
 - use different attribute sets for input
 - use different partitions of class labels
 - use different learning algorithms

Bagging

- Bootstrap sampling: sampling with replacement

Original Data	1	2	3	4	5	6	7	8	9	10
Bagging (Round 1)	7	8	10	8	2	5	10	10	5	9
Bagging (Round 2)	1	4	9	1	2	3	2	7	3	2
Bagging (Round 3)	1	8	5	10	5	5	9	6	3	7

10 distinct items
in dataset

1st sample: 6 distinct items drawn.
Item 10 is picked 3 times

- Build a classifier on each bootstrap sample
- The classifiers obtained from the rounds form an ensemble of classifiers
- Probability of a training instance being selected in a bootstrap sample is:
 - $1 - \left(1 - \frac{1}{n}\right)^n$ (n : number of training instances)
 - ~ 0.632 when n is large

Bagging Example

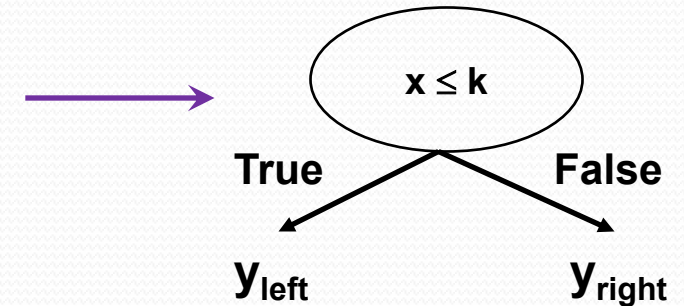


- Consider 1-dimensional data set:

Original Data:

Independent attribute	x	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
Label attribute	y	1	1	1	-1	-1	-1	-1	1	1	1

- Classifier is a decision stump (decision tree of size 1)
 - Decision rule: $(x \leq k)$ vs. $(x > k)$
 - Split point k is chosen based on entropy



Bagging Example

split point

Bagging Round 1:

x	0.1	0.2	0.2	0.3	0.4	0.4	0.5	0.6	0.9	0.9
y	1	1	1	1	-1	-1	-1	-1	1	1

$x \leq 0.35 \rightarrow y = 1$

$x > 0.35 \rightarrow y = -1$

Bagging Round 2:

x	0.1	0.2	0.3	0.4	0.5	0.5	0.9	1	1	1
y	1	1	1	-1	-1	-1	1	1	1	1

$x \leq 0.7 \rightarrow y = 1$

$x > 0.7 \rightarrow y = 1$

Bagging Round 3:

x	0.1	0.2	0.3	0.4	0.4	0.5	0.7	0.7	0.8	0.9
y	1	1	1	-1	-1	-1	-1	-1	1	1

$x \leq 0.35 \rightarrow y = 1$

$x > 0.35 \rightarrow y = -1$

Bagging Round 4:

x	0.1	0.1	0.2	0.4	0.4	0.5	0.5	0.7	0.8	0.9
y	1	1	1	-1	-1	-1	-1	-1	1	1

$x \leq 0.3 \rightarrow y = 1$

$x > 0.3 \rightarrow y = -1$

Bagging Round 5:

x	0.1	0.1	0.2	0.5	0.6	0.6	0.6	1	1	1
y	1	1	1	-1	-1	-1	-1	1	1	1

$x \leq 0.35 \rightarrow y = 1$

$x > 0.35 \rightarrow y = -1$

Bagging Example

Bagging Round 6:

x	0.2	0.4	0.5	0.6	0.7	0.7	0.7	0.8	0.9	1
y	1	-1	-1	-1	-1	-1	-1	1	1	1

$x \leq 0.75 \rightarrow y = -1$

$x > 0.75 \rightarrow y = 1$

Bagging Round 7:

x	0.1	0.4	0.4	0.6	0.7	0.8	0.9	0.9	0.9	1
y	1	-1	-1	-1	-1	1	1	1	1	1

$x \leq 0.75 \rightarrow y = -1$

$x > 0.75 \rightarrow y = 1$

Bagging Round 8:

x	0.1	0.2	0.5	0.5	0.5	0.7	0.7	0.8	0.9	1
y	1	1	-1	-1	-1	-1	-1	1	1	1

$x \leq 0.75 \rightarrow y = -1$

$x > 0.75 \rightarrow y = 1$

Bagging Round 9:

x	0.1	0.3	0.4	0.4	0.6	0.7	0.7	0.8	1	1
y	1	1	-1	-1	-1	-1	-1	1	1	1

$x \leq 0.75 \rightarrow y = -1$

$x > 0.75 \rightarrow y = 1$

Bagging Round 10:

x	0.1	0.1	0.1	0.1	0.3	0.3	0.8	0.8	0.9	0.9
y	1	1	1	1	1	1	1	1	1	1

$x \leq 0.05 \rightarrow y = 1$

$x > 0.05 \rightarrow y = 1$

Bagging Example

- Summary of Trained Decision Stumps:

Round	Split Point	Left Class	Right Class
1	0.35	1	-1
2	0.7	1	1
3	0.35	1	-1
4	0.3	1	-1
5	0.35	1	-1
6	0.75	-1	1
7	0.75	-1	1
8	0.75	-1	1
9	0.75	-1	1
10	0.05	1	1

(A committee of fools is better than a lone genius)

Bagging Example

10 classifiers

Round	x=0.1	x=0.2	x=0.3	x=0.4	x=0.5	x=0.6	x=0.7	x=0.8	x=0.9	x=1.0
1	1	1	1	-1	-1	-1	-1	-1	-1	-1
2	1	1	1	1	1	1	1	1	1	1
3	1	1	1	-1	-1	-1	-1	-1	-1	-1
4	1	1	1	-1	-1	-1	-1	-1	-1	-1
5	1	1	1	-1	-1	-1	-1	-1	-1	-1
6	-1	-1	-1	-1	-1	-1	-1	1	1	1
7	-1	-1	-1	-1	-1	-1	-1	1	1	1
8	-1	-1	-1	-1	-1	-1	-1	1	1	1
9	-1	-1	-1	-1	-1	-1	-1	1	1	1
10	1	1	1	1	1	1	1	1	1	1
Sum	2	2	2	-6	-6	-6	-6	2	2	2
Predicted Class	1	1	1	-1	-1	-1	-1	1	1	1

net votes

majority prediction

+2: there are two more +ve votes than -ve votes for x = 0.1

classify (x = 0.1) as +ve class

-6: there are six more -ve votes than +ve votes for x = 0.6

classify (x = 0.6) as -ve class

Boosting

- An iterative procedure to adaptively change distribution of training data by focusing more on previously misclassified records
 - Initially, all N labeled records are assigned equal weights (for being selected for training)
 - Unlike bagging, weights may change at the end of each boosting round

Boosting

- Records that were wrongly classified in the previous round will have their weights increased in the next round
- Records that were classified correctly in the previous round will have their weights decreased in the next round

Original Data	1	2	3	4	5	6	7	8	9	10
Boosting (Round 1)	7	3	2	8	7	9	4	10	6	3
Boosting (Round 2)	5	4	9	4	2	5	1	7	4	2
Boosting (Round 3)	4	4	8	10	4	5	4	6	3	4

} sample selected in each round

- Suppose example (data object) 4 is hard to classify (that it was incorrectly classified in Rounds 1 & 2)
- Its weight is increased; Therefore, it is more likely to be chosen again in subsequent rounds

Random Forest Algorithm

- Construct an ensemble of decision trees by manipulating training set as well as features
 - Use bootstrap sample to train every decision tree (similar to Bagging)
 - Use the following tree induction algorithm:
 - At every internal node of decision tree, randomly sample p attributes for selecting split criterion